

Hi3516 设备人脸 HTTP-SDK 第三方平台对接使用手册

版本：V1.0

适用范围：Hi3516 设备人脸抓拍配置、人脸库管理、人脸信息管理、人脸抓拍/比对事件 HTTP 推送

修订日期：2026-05-14

1. 对接目标

第三方平台通过本文档可以完成以下工作：

1. 通过 HTTP 向设备下发人脸相关命令。
2. 通过 HTTP 接收设备主动推送的人脸抓拍事件。
3. 通过 HTTP 接收设备主动推送的人脸比对事件。
4. 使用 SDK 命令名完成业务对接，但不需要直接链接 SDK 动态库。

2. 第三方平台需要准备什么

第三方平台需要准备两个能力：

能力	平台职责	示例
命令 HTTP 客户端	主动调用设备 <code>/api/v1/sdk/command</code>	添加人脸、获取目标库、设置抓拍配置
事件 HTTP 服务端	提供回调接口让设备主动 POST	<code>http://<platform-ip>:18080/face/event</code>

推荐联调步骤：

1. 确认平台可以访问设备 HTTP 端口。
2. 平台启动事件回调服务，例如监听 18080 端口。
3. 配置设备 HTTP 推送地址为平台回调 URL。
4. 平台调用命令接口，完成目标库、人脸、抓拍配置设置。
5. 触发设备人脸抓拍或人脸比对，检查平台是否收到推送。

3. 地址、端口和路径

3.1 设备命令地址

设备命令统一使用 HTTP POST：

```
http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
```

如果项目现场 CGI 名称或 Web 路径不同，只需要保证请求路径最终以以下后缀结尾：

```
/api/v1/sdk/command
```

例如下面这些形式都属于同一类入口：

```
http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command
http://192.168.1.100/cgi-bin/xxx/api/v1/sdk/command
http://192.168.1.100:8088/cgi-bin/encoder.cgi/api/v1/sdk/command
```

3.2 端口说明

端口/地址	方向	是否第三方需要访问	说明
设备 HTTP 端口，通常 80	平台 -> 设备	是	第三方平台下发命令时访问。现场如果改过 Web 端口，以现场为准。
/api/v1/sdk/command	平台 -> 设备	是	HTTP-SDK 命令统一入口。
8080	设备内部	否	CGI 转发到内部业务 shortLink 的端口，不对第三方开放。
平台回调端口，示例 18080	设备 -> 平台	是	平台自己监听，用来接收设备推送。
SDK HTTP server demo 端口，示例 9000	Demo	仅测试	用于本地模拟设备。生产环境不要按 9000 理解设备端口。

3.3 平台回调地址

平台需要提供一个可被设备访问的 HTTP 地址，例如：

```
http://<platform-ip>:18080/face/event
```

设备将使用 POST multipart/form-data 向该地址推送事件。

4. 通用命令协议

4.1 HTTP 方法和请求头

所有命令统一使用 `POST`。

```
POST /cgi-bin/encoder.cgi/api/v1/sdk/command HTTP/1.1
Host: <device-ip>
Content-Type: application/json
```

如果设备 Web 服务启用了鉴权，请按照设备现有 HTTP 鉴权方式增加认证头。HTTP-SDK 命令体本身不包含登录会话。

4.2 请求体通用格式

```
{
  "Command": "NET_TV_GET_TARGET_LIB",
  "Data": {}
}
```

字段说明：

字段	类型	必填	说明
Command	string/number	是	SDK 命令名或命令码。建议使用字符串命令名。
Data	object	否	业务参数。无参数命令传 {}。

兼容字段：

字段	说明
SdkCommand	可替代 Command。
SdkCommandName	可替代 Command。

4.3 响应通用格式

典型响应：

```
{
  "ActionCode": 7203,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {}
}
```

字段说明：

字段	说明
ActionCode	设备内部业务码，第三方平台一般只用于日志定位。
Return	0 表示业务成功，非 0 表示失败。
Data	业务返回数据。不同命令结构不同。

注意：

1. HTTP 状态码 200 只表示 CGI 层返回成功。
2. 业务是否成功请看响应体中的 Return。
3. 如果 HTTP 超时或连接失败，平台应按网络失败处理，可重试。

5. 第三方平台 C++ 通用调用模板

第三方可以使用任意 HTTP 客户端库。下面给出基于 `cpp-http-lib` 风格的最小模板，SDK demo 中也有类似实现：

```
SDK/af_sdk/sdk_client/demo/http_face/main.cpp
```

5.1 通用请求函数

```
#include <iostream>
#include <string>
#include "tvsdkhttplib.h"

namespace httpLib = tvsdk::httpLib;

struct HttpResult
{
    int status = 0;
    std::string body;
};

std::string BuildSdkCommandBody(const std::string &command,
                                const std::string &dataJson)
{
    const std::string data = dataJson.empty() ? "{}" : dataJson;
    return std::string("{\"Command\":\"" + command + "\",\"Data\":\"" + data
+ "\"}");
}

HttpResult PostSdkCommand(const std::string &deviceBaseUrl,
                           const std::string &command,
                           const std::string &dataJson)
{
    HttpResult result;

    httpLib::Client cli(deviceBaseUrl);
```

```

cli.set_connection_timeout(5, 0);
cli.set_read_timeout(10, 0);
cli.set_write_timeout(10, 0);

const std::string body = BuildSdkCommandBody(command, dataJson);
auto res = cli.Post("/api/v1/sdk/command", body, "application/json");
if (!res)
{
    std::cerr << "HTTP request failed, command=" << command <<
std::endl;
    return result;
}

result.status = res->status;
result.body = res->body;
return result;
}

```

调用时:

```

HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_GET_TARGET_LIB",
    "{}");

std::cout << "HTTP=" << ret.status << "\n" << ret.body << std::endl;

```

如果第三方使用的 HTTP 库不支持 `deviceBaseUrl` 中带 CGI 路径, 请直接请求完整 URL:

```
http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command
```

6. 命令总表

序号	功能	SDK 命令名	SDK 命令码	内部 ActionCode	HTTP 方法
1	获取人脸抓拍配置	NET_TV_GET_FACECAPTUREINFO	246	2518	POST
2	设置人脸抓拍配置	NET_TV_SET_FACECAPTUREINFO	247	2519	POST
3	获取人脸比对配置	NET_TV_GET_FACE_COMPARE_INFO	491	2529	POST
4	设置人脸比对配置	NET_TV_SET_FACE_COMPARE_INFO	482	2528	POST
5	添加人脸	NET_TV_ADD_FACE_INFO	487	7204	POST
6	删除人脸	NET_TV_DEL_FACE_INFO	488	7205	POST
7	修改人脸	NET_TV_SET_FACE_INFO	489	7206	POST
8	获取人脸	NET_TV_GET_FACE_INFO	490	7207	POST

序号	功能	SDK 命令名	SDK 命令码	内部 ActionCode	HTTP 方法
9	添加目标库	NET_TV_ADD_TARGET_LIB	483	7200	POST
10	删除目标库	NET_TV_DEL_TARGET_LIB	484	7201	POST
11	修改目标库	NET_TV_SET_TARGET_LIB	485	7202	POST
12	获取目标库	NET_TV_GET_TARGET_LIB	486	7203	POST

字段命名重点：

1. 当前设备中 LibId 表示目标库名称，例如 员工库。
2. 删除人脸使用 Ids 数组。
3. 修改人脸使用 Id，不是 FaceID。
4. 添加人脸时 PicPath 是设备本地图片路径，不是平台本地路径。

7. 命令 1：获取人脸抓拍配置

7.1 使用场景

平台在展示设备当前人脸抓拍配置前调用该接口，例如页面打开时读取：

1. 人脸抓拍是否启用。
2. 抓拍间隔。
3. 检测区域、屏蔽区域。
4. 联动方式。

7.2 请求信息

项目	内容
URL	http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
Method	POST
Content-Type	application/json
Command	NET_TV_GET_FACECAPTUREINFO

7.3 请求JSON

```
{
  "Command": "NET_TV_GET_FACECAPTUREINFO",
  "Data": {}
}
```

7.4 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_GET_FACECAPTUREINFO",
  "Data": {}
}'
```

7.5 C++ 调用示例

```
HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_GET_FACECAPTUREINFO",
    "{}");
```

7.6 成功响应示例

```
{
  "ActionCode": 2518,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {
    "Enable": true,
    "Rule": {
      "Sensitivity": 50,
      "Interval": 5,
      "MinWidth": 30,
      "MinHeight": 30,
      "MaxWidth": 225,
      "MaxHeight": 225
    },
    "LinkageMode": {
      "Tradition": [
        6,
        7
      ],
      "AlarmLinkage": [],
      "RecordChn": []
    }
  }
}
```

```
}  
}  
}
```

7.7 平台处理建议

1. `Return=0` 时解析 `Data`。
2. 如果某些配置字段不存在，平台应按默认值展示，避免页面异常。
3. 区域、布防时间等复杂字段以设备返回为准，不建议平台自行构造未知结构。

8. 命令 2：设置人脸抓拍配置

8.1 使用场景

平台修改设备人脸抓拍规则时调用该接口，例如：

1. 开启或关闭人脸抓拍。
2. 修改抓拍间隔。
3. 修改抓拍尺寸限制。
4. 修改联动方式，例如是否上传全景图、目标图。

8.2 请求信息

项目	内容
URL	<code>http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command</code>
Method	<code>POST</code>
Content-Type	<code>application/json</code>
Command	<code>NET_TV_SET_FACECAPTUREINFO</code>

8.3 请求 JSON

```
{  
  "Command": "NET_TV_SET_FACECAPTUREINFO",  
  "Data": {  
    "Enable": true,  
    "Rule": {  
      "Sensitivity": 50,  
      "Interval": 5,  
      "MinWidth": 30,  
      "MinHeight": 30,  
      "MaxWidth": 225,  
      "MaxHeight": 225  
    }  
  }  
}
```

```
    },
    "LinkageMode": {
      "Tradition": [
        6,
        7
      ],
      "AlarmLinkage": [],
      "RecordChn": []
    }
  }
}
```

8.4 字段说明

字段	类型	必填	说明
Enable	bool	是	是否启用人脸抓拍。
Rule.Sensitivity	number	否	灵敏度，建议 0-100。
Rule.Interval	number	否	抓拍间隔，单位秒。
Rule.Minwidth	number	否	最小目标宽度。
Rule.MinHeight	number	否	最小目标高度。
Rule.Maxwidth	number	否	最大目标宽度。
Rule.MaxHeight	number	否	最大目标高度。
LinkageMode.Tradition	array	否	传统联动类型数组。

常用联动值：

值	说明
1	发送邮件
3	上传 SD 卡
6	上传人脸全景图
7	上传人脸目标图

8.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_SET_FACECAPTUREINFO",
  "Data": {
```

```

    "Enable": true,
    "Rule": {
        "Sensitivity": 50,
        "Interval": 5,
        "Minwidth": 30,
        "MinHeight": 30,
        "Maxwidth": 225,
        "MaxHeight": 225
    },
    "LinkageMode": {
        "Tradition": [6, 7],
        "AlarmLinkage": [],
        "RecordChn": []
    }
}
}'

```

8.6 C++ 调用示例

```

const std::string data = R"({
    "Enable": true,
    "Rule": {
        "Sensitivity": 50,
        "Interval": 5,
        "Minwidth": 30,
        "MinHeight": 30,
        "Maxwidth": 225,
        "MaxHeight": 225
    },
    "LinkageMode": {
        "Tradition": [6, 7],
        "AlarmLinkage": [],
        "RecordChn": []
    }
})";

HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_SET_FACECAPTUREINFO",
    data);

```

8.7 成功响应示例

```

{
    "ActionCode": 2519,
    "DeviceName": "IPC",
    "UserName": "admin",
    "Return": 0,
    "Data": {}
}

```

8.8 注意事项

1. 如果平台只想修改部分字段，建议先调用“获取人脸抓拍配置”，在原配置基础上修改后再整体下发。
2. 检测区域、屏蔽区域、布防时间属于复杂配置，第三方需要按设备返回结构原样维护。
3. 如果开启 HTTP 推送，还需要同时配置设备 HTTP 推送地址，见“设备向平台推送配置”章节。

9. 命令 3：获取人脸比对配置

9.1 使用场景

平台在展示设备当前人脸比对配置前调用该接口，例如页面打开时读取：

1. 人脸比对是否启用。
2. 比对成功后的联动方式。
3. 比对失败后的联动方式。
4. 人脸比对布防时间。

9.2 请求信息

项目	内容
URL	<code>http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command</code>
Method	<code>POST</code>
Content-Type	<code>application/json</code>
Command	<code>NET_TV_GET_FACE_COMPARE_INFO</code>

9.3 请求 JSON

```
{
  "Command": "NET_TV_GET_FACE_COMPARE_INFO",
  "Data": {}
}
```

9.4 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_GET_FACE_COMPARE_INFO",
  "Data": {}
}'
```

9.5 C++ 调用示例

```
HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_GET_FACE_COMPARE_INFO",
    "{}");
```

9.6 成功响应示例

```
{
  "ActionCode": 2529,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {
    "Enable": true,
    "LinkageSuccessMode": {
      "Tradition": [
        6,
        7
      ],
      "AlarmLinkage": [],
      "RecordChn": []
    },
    "LinkageFailMode": {
      "Tradition": [
        6
      ],
      "AlarmLinkage": [],
      "RecordChn": []
    }
  }
}
```

9.7 平台处理建议

1. `LinkageSuccessMode` 表示比对成功后触发的联动。
2. `LinkageFailMode` 表示比对失败后触发的联动。
3. 平台修改配置时建议先获取原配置，再只调整需要变更的字段后整体下发。

10. 命令 4：设置人脸比对配置

10.1 使用场景

平台修改设备人脸比对规则时调用该接口，例如：

1. 开启或关闭人脸比对。
2. 设置比对成功联动，例如上传全景图、目标图。
3. 设置比对失败联动，例如只上传抓拍图。
4. 设置人脸比对布防时间。

10.2 请求信息

项目	内容
URL	<code>http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command</code>
Method	POST
Content-Type	<code>application/json</code>
Command	<code>NET_TV_SET_FACE_COMPARE_INFO</code>

10.3 请求 JSON

```
{
  "Command": "NET_TV_SET_FACE_COMPARE_INFO",
  "Data": {
    "Enable": true,
    "LinkageSuccessMode": {
      "Tradition": [
        6,
        7
      ],
      "AlarmLinkage": [],
      "RecordChn": []
    },
    "LinkageFailMode": {
      "Tradition": [
        6
      ],
      "AlarmLinkage": [],
      "RecordChn": []
    }
  }
}
```

```
}  
}
```

10.4 字段说明

字段	类型	必填	说明
Enable	bool	是	是否启用人脸比对。
LinkageSuccessMode	object	否	比对成功联动配置。
LinkageSuccessMode.Tradition	array	否	比对成功传统联动类型数组。
LinkageFailMode	object	否	比对失败联动配置。
LinkageFailMode.Tradition	array	否	比对失败传统联动类型数组。

常用联动值：

值	说明
1	发送邮件
3	上传 SD 卡
6	上传人脸全景图
7	上传人脸目标图

10.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command"  
\br/>-H "Content-Type: application/json" \  
-d '{  
  "Command": "NET_TV_SET_FACE_COMPARE_INFO",  
  "Data": {  
    "Enable": true,  
    "LinkageSuccessMode": {  
      "Tradition": [6, 7],  
      "AlarmLinkage": [],  
      "RecordChn": []  
    },  
    "LinkageFailMode": {  
      "Tradition": [6],  
      "AlarmLinkage": [],  
      "RecordChn": []  
    }  
  }  
}'
```

10.6 C++ 调用示例

```
const std::string data = R"({
    "Enable": true,
    "LinkageSuccessMode": {
        "Tradition": [6, 7],
        "AlarmLinkage": [],
        "RecordChn": []
    },
    "LinkageFailMode": {
        "Tradition": [6],
        "AlarmLinkage": [],
        "RecordChn": []
    }
})";

HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_SET_FACE_COMPARE_INFO",
    data);
```

10.7 成功响应示例

```
{
    "ActionCode": 2528,
    "DeviceName": "IPC",
    "UserName": "admin",
    "Return": 0,
    "Data": {}
}
```

10.8 注意事项

1. 人脸比对依赖目标库和人员底图，建议先完成目标库、人脸添加，再开启比对。
2. 如果平台只修改联动配置，建议先调用“获取人脸比对配置”后在原配置基础上修改。
3. NET_TV_SET_FACE_COMPARE_INFO 的命令码为 482，这是已有 SDK 命令码；新增的获取命令为 491，不会影响原有目标库和人脸命令码。

11. 命令 5：添加人脸

11.1 使用场景

平台向设备目标库中新增一名人员，例如向 员工库 添加人员 张三。

11.2 请求信息

项目	内容
URL	http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
Method	POST
Content-Type	application/json
Command	NET_TV_ADD_FACE_INFO

11.3 请求 JSON

```
{
  "Command": "NET_TV_ADD_FACE_INFO",
  "Data": {
    "LibId": "员工库",
    "Name": "张三",
    "PhoneNum": "13800000000",
    "PicPath": "/opt/cam/face/zhangsan.jpg",
    "BinPath": "",
    "PicType": "jpg",
    "PicSize": 102400,
    "PicDate": "2026-05-14 10:00:00",
    "Picwidth": 640,
    "PicHeight": 480
  }
}
```

11.4 字段说明

字段	类型	必填	说明
LibId	string	是	目标库名称，例如 员工库。该目标库需要先创建。
Name	string	是	人员姓名。
PhoneNum	string	否	联系方式。
PicPath	string	是	设备本地图片路径，不是平台 PC 本地路径。
BinPath	string	否	特征文件路径，可为空。
PicType	string	否	图片类型，例如 jpg、jpeg。
PicSize	number	否	图片大小，单位字节。
PicDate	string	否	图片时间。
Picwidth	number	否	图片宽度。
PicHeight	number	否	图片高度。

11.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_ADD_FACE_INFO",
  "Data": {
    "LibId": "员工库",
    "Name": "张三",
    "PhoneNum": "13800000000",
    "PicPath": "/opt/cam/face/zhangsan.jpg",
    "BinPath": "",
    "PicType": "jpg",
    "PicSize": 102400,
    "PicDate": "2026-05-14 10:00:00",
    "Picwidth": 640,
    "PicHeight": 480
  }
}'
```

11.6 C++ 调用示例

```
const std::string data = R"({
  "LibId": "员工库",
  "Name": "张三",
  "PhoneNum": "13800000000",
  "PicPath": "/opt/cam/face/zhangsan.jpg",
  "BinPath": "",
```

```
"PicType": "jpg",
"PicSize": 102400,
"PicDate": "2026-05-14 10:00:00",
"Picwidth": 640,
"PicHeight": 480
}));

HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_ADD_FACE_INFO",
    data);
```

11.7 成功响应示例

```
{
  "ActionCode": 7204,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {}
}
```

11.8 注意事项

1. `PicPath` 必须是设备上已经存在的图片路径。
2. 当前命令不是图片上传接口，不支持把平台图片文件直接放入 JSON。
3. 如果平台只有图片二进制，需要先通过设备已有文件上传通道、SFTP、运维脚本或后续扩展接口将图片放到设备本地。
4. 添加目标库后再添加人脸，否则人脸可能无法写入对应库。

12. 命令 6：删除人脸

12.1 使用场景

平台从设备目标库中删除一个或多个人员。

12.2 请求信息

项目	内容
URL	<code>http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command</code>
Method	<code>POST</code>
Content-Type	<code>application/json</code>
Command	<code>NET_TV_DEL_FACE_INFO</code>

12.3 请求JSON

```
{
  "Command": "NET_TV_DEL_FACE_INFO",
  "Data": {
    "Ids": [
      10001,
      10002
    ]
  }
}
```

12.4 字段说明

字段	类型	必填	说明
Ids	number array	是	要删除的人脸记录 ID 列表。

12.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_DEL_FACE_INFO",
  "Data": {
    "Ids": [10001, 10002]
  }
}'
```

12.6 C++ 调用示例

```
const std::string data = R"({
  "Ids": [10001, 10002]
})";

HttpResult ret = PostSdkCommand(
  "http://192.168.1.100/cgi-bin/encoder.cgi",
  "NET_TV_DEL_FACE_INFO",
  data);
```

12.7 成功响应示例

```
{
  "ActionCode": 7205,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {}
}
```

12.8 注意事项

1. 删除人脸使用 `Ids`，不是 `FaceID`。
2. `Ids` 必须是数组，即使只删除一个人员也建议传数组。
3. 删除后如平台仍缓存人员信息，需要同步清理平台缓存。

13. 命令 7：修改人脸

13.1 使用场景

平台修改设备中已有人员的信息，例如修改姓名、手机号或图片路径。

13.2 请求信息

项目	内容
URL	<code>http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command</code>
Method	<code>POST</code>
Content-Type	<code>application/json</code>
Command	<code>NET_TV_SET_FACE_INFO</code>

13.3 请求 JSON

```
{
  "Command": "NET_TV_SET_FACE_INFO",
  "Data": {
    "Id": 10001,
    "LibId": "员工库",
    "Name": "张三-修改",
    "PhoneNum": "13900000000",
    "PicPath": "/opt/cam/face/zhangsan_new.jpg",
    "PicType": "jpg",
    "PicSize": 120000,
    "PicDate": "2026-05-14 11:00:00"
  }
}
```

13.4 字段说明

字段	类型	必填	说明
<code>Id</code>	number	是	人脸记录 ID。
<code>LibId</code>	string	是	目标库名称。
<code>Name</code>	string	否	人员姓名。
<code>PhoneNum</code>	string	否	联系方式。
<code>PicPath</code>	string	否	新图片在设备本地的路径。
<code>PicType</code>	string	否	图片类型。
<code>PicSize</code>	number	否	图片大小。
<code>PicDate</code>	string	否	图片时间。

13.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_SET_FACE_INFO",
  "Data": {
    "Id": 10001,
    "LibId": "员工库",
    "Name": "张三-修改",
    "PhoneNum": "13900000000",
    "PicPath": "/opt/cam/face/zhangsan_new.jpg",
    "PicType": "jpg",
    "PicSize": 120000,
    "PicDate": "2026-05-14 11:00:00"
  }
}
```

```
}'
```

13.6 C++ 调用示例

```
const std::string data = R"({
    "Id": 10001,
    "LibId": "员工库",
    "Name": "张三-修改",
    "PhoneNum": "13900000000",
    "PicPath": "/opt/cam/face/zhangsan_new.jpg",
    "PicType": "jpg",
    "PicSize": 120000,
    "PicDate": "2026-05-14 11:00:00"
})";

HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_SET_FACE_INFO",
    data);
```

13.7 成功响应示例

```
{
    "ActionCode": 7206,
    "DeviceName": "IPC",
    "UserName": "admin",
    "Return": 0,
    "Data": {}
}
```

13.8 注意事项

1. 修改人脸使用 `Id`，不是 `FaceID`。
2. 如果修改图片，新的 `PicPath` 也必须是设备本地可访问路径。
3. 修改后设备可能需要重新建模或更新特征，平台应以查询结果中的 `ModelState` 为准。

14. 命令 8：获取人脸

14.1 使用场景

平台按条件查询设备中某个目标库的人脸记录，例如：

1. 查询 `员工库` 中所有人员。
2. 按姓名模糊查询。
3. 按手机号模糊查询。
4. 按建模状态筛选。

14.2 请求信息

项目	内容
URL	http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
Method	POST
Content-Type	application/json
Command	NET_TV_GET_FACE_INFO

14.3 请求 JSON

```
{
  "Command": "NET_TV_GET_FACE_INFO",
  "Data": {
    "LibId": "员工库",
    "Name": "",
    "PhoneNum": "",
    "ModelState": -1,
    "RatingLevel": -1
  }
}
```

14.4 查询条件说明

字段	类型	必填	说明
LibId	string	建议必填	目标库名称。当前设备按库表查询，建议必须传。
Name	string	否	姓名模糊匹配，空字符串表示不过滤。
PhoneNum	string	否	手机号模糊匹配，空字符串表示不过滤。
ModelState	number	否	-1 不过滤；0 未建模；1 成功；2 失败。
RatingLevel	number	否	-1 不过滤；0 评分未知；1 低分段；8 高分段。

14.5 curl 示例

```

curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_GET_FACE_INFO",
  "Data": {
    "LibId": "员工库",
    "Name": "",
    "PhoneNum": "",
    "ModelState": -1,
    "RatingLevel": -1
  }
}'

```

14.6 C++ 调用示例

```

const std::string data = R"({
  "LibId": "员工库",
  "Name": "",
  "PhoneNum": "",
  "ModelState": -1,
  "RatingLevel": -1
})";

HttpResult ret = PostSdkCommand(
  "http://192.168.1.100/cgi-bin/encoder.cgi",
  "NET_TV_GET_FACE_INFO",
  data);

```

14.7 成功响应示例

```

{
  "ActionCode": 7207,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {
    "FaceInfos": [
      {
        "Id": 10001,
        "LibId": "员工库",
        "Name": "张三",
        "PhoneNum": "13800000000",
        "PicPath": "/opt/cam/face/zhangsan.jpg",
        "PicType": "jpg",
        "PicSize": 102400,
        "PicDate": "2026-05-14 10:00:00",
        "ModelState": 1,
        "RatingLevel": 8,

```

```
        "BinPath": "/opt/cam/face/zhangsan.bin"
      }
    ]
  }
}
```

14.8 平台处理建议

1. 平台应保存 `Id`，后续删除和修改人脸都需要使用该字段。
2. 如果 `FaceInfos` 为空，表示没有符合条件的记录。
3. `ModelState` 非成功时，平台可展示为“建模中”或“建模失败”。

15. 命令 9：添加目标库

15.1 使用场景

平台创建一个目标库，例如 `员工库`、`访客库`、`黑名单库`。

15.2 请求信息

项目	内容
URL	<code>http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command</code>
Method	<code>POST</code>
Content-Type	<code>application/json</code>
Command	<code>NET_TV_ADD_TARGET_LIB</code>

15.3 请求 JSON

```
{
  "Command": "NET_TV_ADD_TARGET_LIB",
  "Data": {
    "LibId": "员工库"
  }
}
```

15.4 字段说明

字段	类型	必填	说明
<code>LibId</code>	string	是	目标库名称。当前设备使用该字段作为库表名称。

15.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_ADD_TARGET_LIB",
  "Data": {
    "LibId": "员工库"
  }
}'
```

15.6 C++ 调用示例

```
const std::string data = R"({
  "LibId": "员工库"
})";

HttpResult ret = PostSdkCommand(
  "http://192.168.1.100/cgi-bin/encoder.cgi",
  "NET_TV_ADD_TARGET_LIB",
  data);
```

15.7 成功响应示例

```
{
  "ActionCode": 7200,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {}
}
```

15.8 注意事项

1. 目标库名称建议不要包含特殊字符、路径分隔符、SQL 保留字符。
2. 同名库重复添加时，设备返回结果以实际业务处理为准。
3. 添加人脸前必须先创建目标库。

16. 命令 10：删除目标库

16.1 使用场景

平台删除一个或多个目标库。删除目标库通常会删除库内人员记录，请谨慎操作。

16.2 请求信息

项目	内容
URL	http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
Method	POST
Content-Type	application/json
Command	NET_TV_DEL_TARGET_LIB

16.3 删除单个目标库请求 JSON

```
{
  "Command": "NET_TV_DEL_TARGET_LIB",
  "Data": {
    "LibId": "员工库"
  }
}
```

16.4 删除多个目标库请求 JSON

```
{
  "Command": "NET_TV_DEL_TARGET_LIB",
  "Data": {
    "LibId": [
      "员工库",
      "访客库"
    ]
  }
}
```

16.5 字段说明

字段	类型	必填	说明
LibId	string/string array	是	要删除的目标库名称，支持单个字符串或字符串数组。

16.6 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_DEL_TARGET_LIB",
  "Data": {
    "LibId": ["员工库", "访客库"]
  }
}'
```

16.7 C++ 调用示例

```
const std::string data = R"({
  "LibId": ["员工库", "访客库"]
})";

HttpResult ret = PostSdkCommand(
  "http://192.168.1.100/cgi-bin/encoder.cgi",
  "NET_TV_DEL_TARGET_LIB",
  data);
```

16.8 成功响应示例

```
{
  "ActionCode": 7201,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {}
}
```

16.9 注意事项

1. 删除目标库是高风险操作，平台应增加二次确认。
2. 删除后建议立即调用“获取目标库”刷新平台列表。
3. 如果库中人员仍在平台侧缓存，需要同步清理。

17. 命令 11：修改目标库

17.1 使用场景

平台修改目标库名称，例如将 员工库 修改为 员工库-华东区。

17.2 请求信息

项目	内容
URL	http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
Method	POST
Content-Type	application/json
Command	NET_TV_SET_TARGET_LIB

17.3 请求 JSON

```
{
  "Command": "NET_TV_SET_TARGET_LIB",
  "Data": {
    "LibId_old": "员工库",
    "LibId_new": "员工库-华东区"
  }
}
```

17.4 字段说明

字段	类型	必填	说明
LibId_old	string	是	原目标库名称。
LibId_new	string	是	新目标库名称。

17.5 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_SET_TARGET_LIB",
  "Data": {
    "LibId_old": "员工库",
    "LibId_new": "员工库-华东区"
  }
}'
```

17.6 C++ 调用示例

```
const std::string data = R"({
    "LibId_old": "员工库",
    "LibId_new": "员工库-华东区"
})";

HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_SET_TARGET_LIB",
    data);
```

17.7 成功响应示例

```
{
    "ActionCode": 7202,
    "DeviceName": "IPC",
    "UserName": "admin",
    "Return": 0,
    "Data": {}
}
```

17.8 注意事项

1. 修改后人脸记录所属库名也会随库表变化。
2. 平台应同步更新本地库名缓存。
3. 新库名不要与已有库冲突。

18. 命令 12：获取目标库

18.1 使用场景

平台获取设备上已有目标库列表和统计信息，例如：

1. 展示库列表。
2. 查看每个库的人脸数量。
3. 查看建模正常/异常数量。

18.2 请求信息

项目	内容
URL	http://<device-ip>/cgi-bin/encoder.cgi/api/v1/sdk/command
Method	POST
Content-Type	application/json
Command	NET_TV_GET_TARGET_LIB

18.3 请求JSON

```
{
  "Command": "NET_TV_GET_TARGET_LIB",
  "Data": {}
}
```

18.4 curl 示例

```
curl -X POST "http://192.168.1.100/cgi-bin/encoder.cgi/api/v1/sdk/command" \
-H "Content-Type: application/json" \
-d '{
  "Command": "NET_TV_GET_TARGET_LIB",
  "Data": {}
}'
```

18.5 C++ 调用示例

```
HttpResult ret = PostSdkCommand(
    "http://192.168.1.100/cgi-bin/encoder.cgi",
    "NET_TV_GET_TARGET_LIB",
    "{}");
```

18.6 成功响应示例

```
{
  "ActionCode": 7203,
  "DeviceName": "IPC",
  "UserName": "admin",
  "Return": 0,
  "Data": {
    "TargetLibInfos": [
      {
        "LibId": "员工库",
        "TotalFace": 10,
        "NormalNum": 8,
        "AbnormalNum": 2
      },
      {
        "LibId": "访客库",
        "TotalFace": 3,
        "NormalNum": 3,
        "AbnormalNum": 0
      }
    ]
  }
}
```

18.7 返回字段说明

字段	说明
TargetLibInfos	目标库列表。
LibId	目标库名称。
TotalFace	库内人脸总数。
NormalNum	建模正常数量。
AbnormalNum	建模异常数量。

18.8 平台处理建议

1. 页面加载库列表时优先调用该接口。
2. 添加、删除、修改库后，应重新调用该接口刷新。
3. 如果 TargetLibInfos 为空，表示设备暂无目标库。

19. 设备向平台推送配置

设备侧 HTTP 推送配置文件：

```
/opt/cam/.config/user_data/face_http_push.json
```

示例：

```
{
  "Enable": true,
  "url": "http://<platform-ip>:18080/face/event",
  "CaptureUrl": "http://<platform-ip>:18080/face/event",
  "CompareUrl": "http://<platform-ip>:18080/face/event",
  "Token": ""
}
```

字段说明：

字段	类型	说明
Enable	bool	总开关，true 表示启用 HTTP 推送。
url	string	默认推送地址。
CaptureUrl	string	人脸抓拍推送地址，空则使用 url。
CompareUrl	string	人脸比对推送地址，空则使用 url。
Token	string	鉴权 Token，非空时设备推送携带 Authorization: Bearer <Token>。

环境变量覆盖：

环境变量	说明
<code>FACE_HTTP_PUSH_ENABLE</code>	覆盖总开关。
<code>FACE_HTTP_PUSH_URL</code>	覆盖默认推送地址。
<code>FACE_HTTP_CAPTURE_URL</code>	覆盖抓拍推送地址。
<code>FACE_HTTP_COMPARE_URL</code>	覆盖比对推送地址。
<code>FACE_HTTP_PUSH_TOKEN</code>	覆盖鉴权 Token。

平台部署建议：

1. 平台回调地址必须能被设备访问。
2. 如果平台和设备跨网段，需要保证路由、防火墙、NAT 映射正确。
3. 如果平台使用域名，设备必须能解析该域名。
4. 如果平台启用 Token 校验，设备配置中的 `Token` 必须与平台一致。

20. 平台接收人脸抓拍推送

20.1 接口要求

平台提供 HTTP 服务：

```
POST http://<platform-ip>:18080/face/event
Content-Type: multipart/form-data
```

平台成功接收后返回：

```
{
  "Ret": 0
}
```

20.2 抓拍推送字段

字段	类型	是否可能为空	说明
EventType	text	否	固定为 FACE_CAPTURE。
Command	text	否	固定为 NET_TV_ALARM_FACE_CAPTURE。
AlarmType	text	否	固定为 NET_TV_ALARM_FACE_CAPTURE。
AlarmCode	text	否	12290，即 0x3002。
DeviceCode	text	否	设备编码。
Channel	text	否	通道号。
TimestampMs	text	否	Unix 毫秒时间戳。
TargetCount	text	否	目标数量。
Targets[i].Left	text	否	目标框左坐标。
Targets[i].Top	text	否	目标框上坐标。
Targets[i].Right	text	否	目标框右坐标。
Targets[i].Bottom	text	否	目标框下坐标。
Targets[i].Confidence	text	否	检测置信度。
Targets[i].Ipd	text	否	瞳距。
PanoramaImage	file	是	全景图 JPEG。
TargetImages[i]	file	是	目标小图 JPEG。

20.3 抓拍推送样例

```
EventType=FACE_CAPTURE
Command=NET_TV_ALARM_FACE_CAPTURE
AlarmType=NET_TV_ALARM_FACE_CAPTURE
AlarmCode=12290
DeviceCode=IPC001
Channel=0
TimestampMs=1778683144077
TargetCount=1
Targets[0].Left=120
Targets[0].Top=80
Targets[0].Right=360
Targets[0].Bottom=420
Targets[0].Confidence=0.965000
Targets[0].Ipd=64
PanoramaImage=<jpeg file>
TargetImages[0]=<jpeg file>
```

20.4 平台解析建议

1. 先读取 `EventType`，判断是否为 `FACE_CAPTURE`。
2. 根据 `TargetCount` 循环读取 `Targets[i].*`。
3. 图片字段按 multipart 文件保存，不要按普通文本字段处理。
4. `TimestampMs` 为毫秒时间戳，平台可转换为本地时间。

21. 平台接收人脸比对推送

21.1 接口要求

和抓拍推送使用同一个平台回调接口即可：

```
POST http://<platform-ip>:18080/face/event
Content-Type: multipart/form-data
```

平台通过 `EventType` 区分事件类型。

21.2 比对推送字段

字段	类型	是否可能为空	说明
EventType	text	否	固定为 FACE_COMPARE。
Command	text	否	固定为 NET_TV_ALARM_FACE_COMPARE。
AlarmType	text	否	固定为 NET_TV_ALARM_FACE_COMPARE。
AlarmCode	text	否	12295，即 0x3007。
DeviceCode	text	否	设备编码。
Channel	text	否	通道号。
TimestampMs	text	否	Unix 毫秒时间戳。
CompareResult	text	否	1 比对成功，0 比对失败。
FaceID	text	是	人员 ID，失败时可能为 -1。
Similarity	text	否	原始相似度，通常 0.0-1.0。
SimilarityPercent	text	否	百分比整数，例如 87。
FaceName	text	是	人员姓名。
FaceLibName	text	是	目标库名称。
LibFacePath	text	是	库中底图路径。
CaptureFacePath	text	是	抓拍人脸图路径。
CaptureImagePath	text	是	抓拍全景图路径。
CaptureFaceImgLen	text	否	抓拍人脸图二进制长度。
LibFaceImgLen	text	否	库中底图二进制长度。
CaptureFaceImage	file	是	抓拍人脸 JPEG。
LibFaceImage	file	是	库中底图 JPEG。

21.3 比对成功样例

```
EventType=FACE_COMPARE
Command=NET_TV_ALARM_FACE_COMPARE
AlarmType=NET_TV_ALARM_FACE_COMPARE
AlarmCode=12295
DeviceCode=IPC001
Channel=0
TimestampMs=1778683154080
CompareResult=1
FaceID=10001
Similarity=0.876000
SimilarityPercent=87
FaceName=张三
FaceLibName=员工库
LibFacePath=/opt/cam/face/lib/10001.jpg
CaptureFacePath=/tmp/face_compare_capture.jpg
CaptureImagePath=/tmp/face_compare_panorama.jpg
CaptureFaceImgLen=95231
LibFaceImgLen=88211
CaptureFaceImage=<jpeg file>
LibFaceImage=<jpeg file>
```

21.4 比对失败样例

```
EventType=FACE_COMPARE
Command=NET_TV_ALARM_FACE_COMPARE
AlarmType=NET_TV_ALARM_FACE_COMPARE
AlarmCode=12295
DeviceCode=IPC001
Channel=0
TimestampMs=1778683154080
CompareResult=0
FaceID=-1
Similarity=0.320000
SimilarityPercent=32
FaceName=
FaceLibName=
LibFacePath=
CaptureFacePath=/tmp/face_compare_capture.jpg
CaptureImagePath=/tmp/face_compare_panorama.jpg
CaptureFaceImgLen=95231
LibFaceImgLen=0
CaptureFaceImage=<jpeg file>
```

21.5 平台解析建议

1. `CompareResult=1` 时展示人员姓名、人员 ID、库名称和相似度。
2. `CompareResult=0` 时按陌生人或未命中处理。
3. `SimilarityPercent` 更适合界面展示。
4. `Similarity` 更适合算法分析或日志记录。
5. 图片字段可能为空，平台保存前需要判断文件是否存在。

22. 第三方平台 C++ 回调服务示例

下面示例展示平台如何接收设备推送。实际项目中需要将图片保存到平台存储，并把事件写入平台数据库。

```
#include <iostream>
#include "tvsdkhttpplib.h"

namespace httpplib = tvsdk::httpplib;

void StartFaceEventServer(int listenPort)
{
    httpplib::Server server;

    server.Post("/face/event", [](const httpplib::Request &req,
httpplib::Response &res) {
        if (!req.is_multipart_form_data())
        {
            res.status = 400;
            res.set_content("{\"Ret\":-1,\"Message\":\"Content-Type must be
multipart/form-data\"}",
                            "application/json");
            return;
        }

        const std::string eventType = req.form.get_field("EventType");

        if (eventType == "FACE_CAPTURE")
        {
            const std::string deviceCode =
req.form.get_field("DeviceCode");
            const std::string channel = req.form.get_field("Channel");
            const std::string targetCount =
req.form.get_field("TargetCount");

            std::cout << "收到人脸抓拍, device=" << deviceCode
                    << ", channel=" << channel
                    << ", targetCount=" << targetCount << std::endl;

            if (req.form.has_file("PanoramaImage"))
            {
                const auto &file = req.form.get_file("PanoramaImage");
```

```

        std::cout << "全景图大小=" << file.content.size() <<
std::endl;
    }
}
else if (eventType == "FACE_COMPARE")
{
    const std::string result = req.form.get_field("CompareResult");
    const std::string faceId = req.form.get_field("FaceID");
    const std::string name = req.form.get_field("FaceName");
    const std::string libName = req.form.get_field("FaceLibName");
    const std::string similarity =
req.form.get_field("SimilarityPercent");

    std::cout << "收到人脸比对, result=" << result
        << ", faceId=" << faceId
        << ", name=" << name
        << ", lib=" << libName
        << ", similarity=" << similarity << std::endl;

    if (req.form.has_file("CaptureFaceImage"))
    {
        const auto &file = req.form.get_file("CaptureFaceImage");
        std::cout << "抓拍人脸图大小=" << file.content.size() <<
std::endl;
    }
}

res.set_content("{\"Ret\":0}", "application/json");
});

server.listen("0.0.0.0", listenPort);
}

```

23. 推荐业务调用顺序

第三方平台首次接入一台设备时，推荐按以下顺序：

1. 调用 `NET_TV_GET_TARGET_LIB`，读取已有目标库。
2. 如不存在目标库，调用 `NET_TV_ADD_TARGET_LIB` 创建目标库。
3. 将人员图片放到设备本地路径。
4. 调用 `NET_TV_ADD_FACE_INFO` 添加人脸。
5. 调用 `NET_TV_GET_FACE_INFO` 检查人员是否写入成功。
6. 调用 `NET_TV_GET_FACECAPTUREINFO` 读取当前抓拍配置。
7. 调用 `NET_TV_SET_FACECAPTUREINFO` 开启或修改抓拍配置。
8. 调用 `NET_TV_GET_FACE_COMPARE_INFO` 读取当前比对配置。
9. 调用 `NET_TV_SET_FACE_COMPARE_INFO` 开启或修改比对配置。
10. 配置设备 HTTP 推送地址。
11. 平台启动 `/face/event` 回调服务。
12. 触发抓拍或比对，检查平台是否收到推送。

24. 错误处理建议

24.1 HTTP 层错误

情况	平台处理建议
连接失败	检查设备 IP、端口、网络、防火墙。
超时	可重试，建议限制重试次数。
HTTP 404	检查 URL 是否以 <code>/api/v1/sdk/command</code> 结尾。
HTTP 401/403	检查设备 HTTP 鉴权。
HTTP 500	设备内部异常，需要结合设备日志定位。

24.2 业务层错误

情况	平台处理建议
<code>Return != 0</code>	展示失败原因或记录日志。
添加人脸失败	检查目标库是否存在、 <code>PicPath</code> 是否在设备本地存在。
删除人脸失败	检查 <code>Ids</code> 是否正确。
查询为空	检查 <code>LibId</code> 是否正确。
设置比对配置失败	检查设备智能资源是否冲突、配置 JSON 是否完整。
收不到推送	检查设备推送配置、平台监听端口、防火墙、Token。

25. 对接检查清单

上线前逐项确认：

1. 平台可以访问设备 HTTP 端口。
2. 平台请求完整 URL 正确。
3. 所有命令使用 `POST`。
4. 请求头包含 `Content-Type: application/json`。
5. 请求体使用 `Command + Data` 格式。
6. 目标库字段使用 `LibId`。
7. 删除人脸字段使用 `Ids` 数组。
8. 添加人脸前图片已经在设备本地。
9. 平台回调接口可被设备访问。
10. 平台回调接口能解析 `multipart/form-data`。
11. 平台回调成功时返回 HTTP `2xx`。
12. 如启用 Token，平台校验 `Authorization: Bearer <Token>`。
13. 平台日志记录 `DeviceCode`、`EventType`、`TimestampMs`、`Command`。
14. 已测试 `NET_TV_GET_FACE_COMPARE_INFO` 和 `NET_TV_SET_FACE_COMPARE_INFO`。

